

Manual

Documentation

miniLaTeX workbench

May 10, 2026

This page intentionally left blank.

Contents

PALPATINE LAB GUIDE

1	LaTeX Workbench: Purpose and Scope	5
1.1	What this software does	5
1.2	Why it exists	5
1.3	Current MVP implemented	5
2	Internal Architecture	7
2.1	How it works internally	7
2.2	Build and preview workflow	7
2.3	Status bar contract	7
3	Sprint Focus: Git and Navigation Workflow	9
3.1	Sprint objective	9
3.2	Git workflow buttons and layout	9
3.3	Branch ordering by recent activity	9
3.4	Graphical Auto Push	9
3.5	Project explorer / solution explorer	10
3.6	Intentionally postponed	10
3.7	Future Neovim and Copilot integration	10

How to read this guide

This handbook uses visual cues inspired by modern networking textbooks: oversized chapter banners, highlighted section cards, and color-coded command blocks. Follow chapters in sequence for a clean setup, then jump to appendices for fast troubleshooting.

This page intentionally left blank.

1

LaTeX Workbench: Purpose and Scope

PALPATINE LAB GUIDE

Contents

1.1	What this software does	5
1.2	Why it exists	5
1.3	Current MVP implemented	5

1.1 What this software does

`latex-workbench` is a lightweight desktop IDE for writing and building LaTeX documents with a practical developer workflow. It combines an editor, build log, shell panel, Git workflow tools, and project navigation in one PySide6 window.

1.2 Why it exists

The project exists to provide a focused *micro-IDE* for technical writing teams who need:

- fast `.tex` editing,
- reproducible builds with `latexmk` and XeLaTeX,
- straightforward branch-aware Git actions,
- minimal UI complexity compared to full-size IDEs.

1.3 Current MVP implemented

The current MVP already supports:

- opening and saving `.tex` files,
- building with `latexmk -xelatex`,
- opening generated PDF files after successful build,
- shell command execution in-project,
- Git pull/status/checkout/auto-push actions,

- branch list refresh and branch visibility in the status bar,
- project explorer navigation for `.tex` files.

2

Internal Architecture

PALPATINE LAB GUIDE

Contents

2.1	How it works internally	7
2.2	Build and preview workflow	7
2.3	Status bar contract	7

2.1 How it works internally

The application keeps UI and service logic modular.

- **UI layer:** main window, editor, Git panel, terminal panel, log panel, and project explorer.
- **Service layer:** file I/O, build execution, shell execution, and Git command orchestration.

2.2 Build and preview workflow

The build flow is:

1. User saves current file.
2. Build service runs `latexmk` in the file directory.
3. Stdout/stderr streams to the Build Log panel.
4. On success, the generated PDF is opened via desktop integration.
5. Build status is reflected in the status bar as **success** or **failed**.

2.3 Status bar contract

The status bar provides a compact operational summary:

Project: /path/to/project | Branch: main | Git: dirty | Build: success

It updates when projects are opened, branch-related operations run, and build outcomes change.

This page intentionally left blank.

3

Sprint Focus: Git and Navigation Workflow

PALPATINE LAB GUIDE

Contents

3.1	Sprint objective	9
3.2	Git workflow buttons and layout	9
3.3	Branch ordering by recent activity	9
3.4	Graphical Auto Push	9
3.5	Project explorer / solution explorer	10
3.6	Intentionally postponed	10
3.7	Future Neovim and Copilot integration	10

3.1 Sprint objective

This sprint improves day-to-day developer flow around branch awareness, push safety, and project navigation.

3.2 Git workflow buttons and layout

UI actions are separated into:

- **Top toolbar:** Open TeX, Save, Build.
- **Git toolbar/panel:** Pull, Status, Refresh, Branch selector, Checkout, Auto Push.

This avoids scattered actions and keeps Git tasks grouped.

3.3 Branch ordering by recent activity

Branch lists are intentionally **not alphabetical**. Branches are loaded with commit-time sorting so the most recently modified branches are shown first.

3.4 Graphical Auto Push

Auto Push reimplements the repository `autopush` behavior with a confirmation dialog:

1. detect changes with `git status --short --branch`,

2. show changed files before commit,
3. require non-empty commit message,
4. run `git add -A`, one `git commit -m`, then `git push`,
5. stream command output into the log panel.

If there are no changes, Auto Push exits gracefully with an informational message.

3.5 Project explorer / solution explorer

A left sidebar presents the active project tree and opens `.tex` files on click. Build artefacts such as `.aux`, `.log`, `.fls`, `.toc`, `.out`, `.xdv`, and `.mtc*` are hidden by default.

3.6 Intentionally postponed

- embedded PDF preview pane,
- PTY-grade terminal emulation,
- embedded Neovim editing,
- in-editor Copilot completions.

3.7 Future Neovim and Copilot integration

Planned direction:

1. replace or augment the plain text editor with embedded Neovim,
2. connect Copilot-compatible Neovim plugins,
3. keep build/Git/project explorer controls in PySide6,
4. preserve branch-aware status and Auto Push safety checks.